WAT4

Web Application Testing

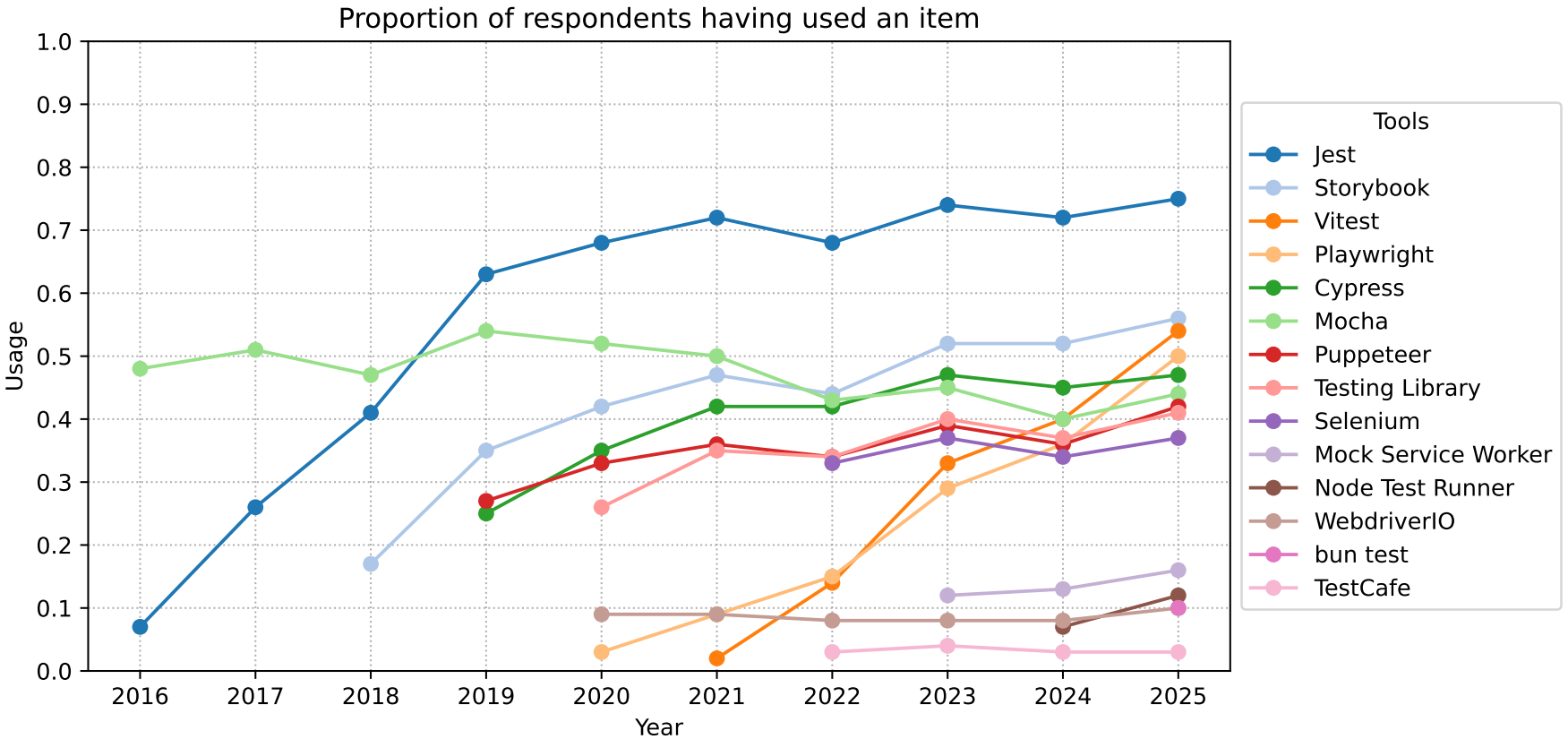
JavaScript Testing with Jest

J. Karder

About (1)

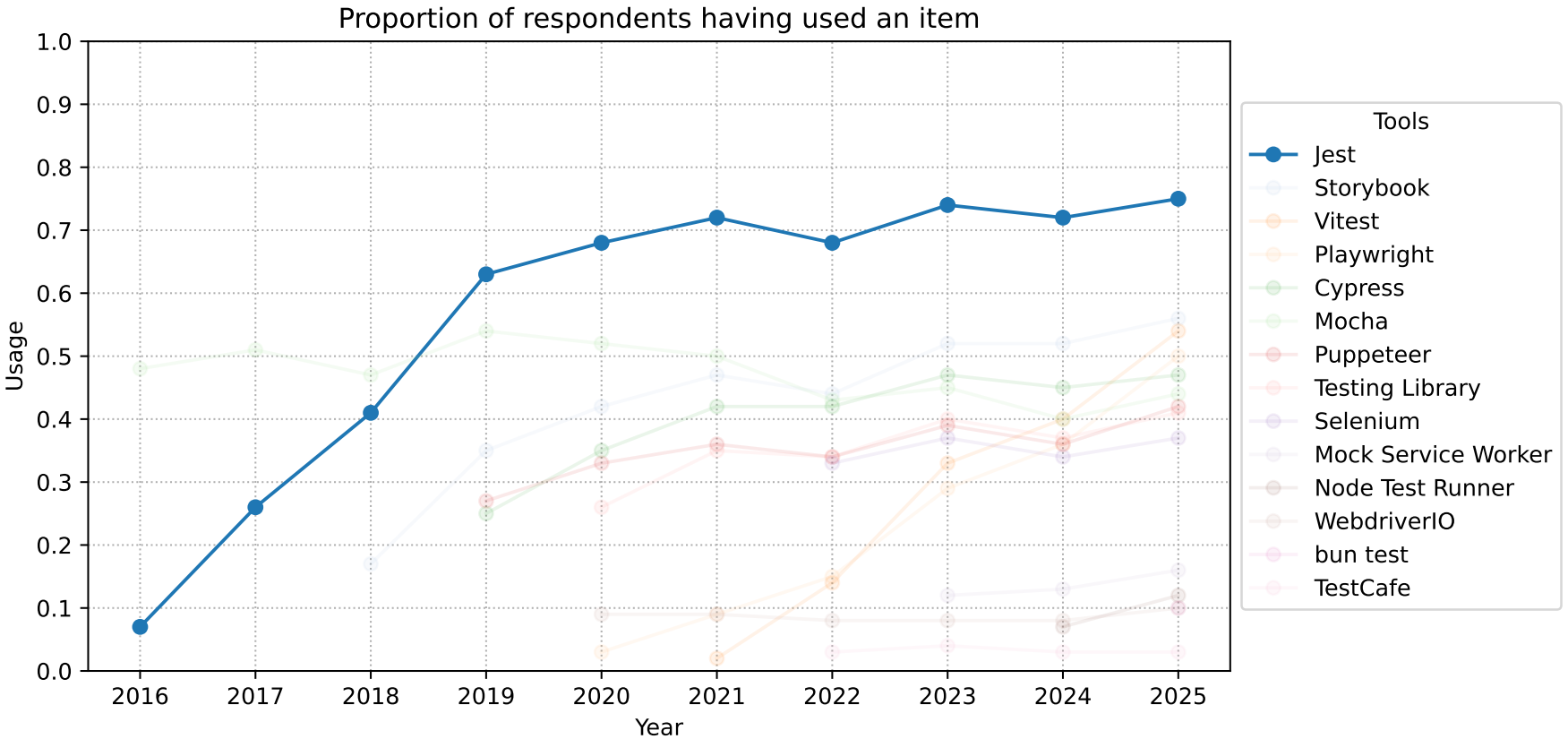
- JavaScript Testing Framework
- Initially created and owned by Meta
  - Development started in 2011
  - Originally called “jst” internally
  - Open-sourced in 2014
  - Jest Open Collective announced by Meta in 2018
  - Since 2018, almost all contributions made by open-source contributors outside of Meta
  - Ownership transferred to OpenJS Foundation in 2022
- GitHub stars: +45k
- GitHub forks: +6k

About (2)



Source: https://2025.stateofjs.com/en-US/libraries/testing/

About (2)



Source: https://2025.stateofjs.com/en-US/libraries/testing/

Getting Started – Add Jest dependency

- Add Jest to `devDependencies` in an existing npm project:

```
npm install --save-dev jest
```

- `package.json` should look like this:

```
{
  // ...
  "devDependencies": {
    // ...
    "jest": "^30.2.0"
    // ...
  }
  // ...
}
```

Getting Started – Add test script

- Add a test script to `package.json` that calls Jest:

```
{
  // ...
  "scripts": {
    "test": "jest"
  }
  // ...
}
```

Getting Started – Implement some function

- Create `sum.js`:

```
function sum(a, b) {
  return a + b;
}
module.exports = sum;
```

Getting Started – Implement test

- Create `sum.test.js`:

```
const sum = require('./sum');

test('adds 1 + 2 to equal 3', () => {
  expect(sum(1, 2)).toBe(3);
});
```

Getting Started – Run tests

- Run the test by calling the previously created npm test script:

```
npm test
```

- Observe Jest output:

```
> jest-demo@1.0.0 test
> jest

 PASS  ./sum.test.js
  ✓ adds 1 + 2 to equal 3 (1 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        0.184 s, estimated 1 s
Ran all test suites.
```

Getting Started – Run tests with coverage

- Add `--coverage` flag when calling Jest:

```
npm test -- --coverage # calls 'jest --coverage'
```

- Observe Jest output:

```
> jest-demo@1.0.0 test
> jest --coverage

 PASS  ./sum.test.js
  √ adds 1 + 2 to equal 3 (1 ms)

----------|---------|----------|---------|---------|-------------------
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
----------|---------|----------|---------|---------|-------------------
All files |     100 |      100 |     100 |     100 |
 sum.js   |     100 |      100 |     100 |     100 |
----------|---------|----------|---------|---------|-------------------
Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        0.33 s, estimated 1 s
Ran all test suites.
```

Configuring Jest

- Recommended via dedicated JavaScript, TypeScript or JSON file
  - Auto-discovered when named `jest.config.js|ts|mjs|cjs|cts|json`
  - Use `--config` flag to pass file explicitly
  - File should export config object:

`jest.config.js`:

```
/** @type {import('jest').Config} */
const config = {
  collectCoverage: true,
  verbose: true
};

module.exports = config;
```

Configuring Jest – `maxConcurrency`

- `maxConcurrency` `[number]`
  - Limits the number of tests that are allowed to run at the same time when using `test.concurrent`
  - Default: 5
  - Any test above this limit will be queued and executed once a slot is released
  - Example:
    - `maxConcurrency: 4`

Configuring Jest – `maxWorkers`

- `maxWorkers` `[number | string]`
  - Specifies the maximum number of workers the worker-pool will spawn for running tests
  - Default:
    - In single run mode: number of the cores available on the machine minus one for the main thread
    - In watch mode: half of the available cores on the machine
  - It may be useful to adjust this in resource limited environments like CIs but the defaults should be adequate for most use-cases.
  - Percentage-based configuration can be used
    - Useful for environments with variable CPUs
  - Examples:
    - `maxWorkers: 4`
    - `maxWorkers: '50%'`

Configuring Jest – `testRegex`

- `testRegex [string | array<string>]`
  - Defines the pattern or patterns Jest uses to detect test files
  - Default: `"(/__tests__/.*|(\\.|/)(test|spec))\\.[mc]?[jt]sx?$"`
    - By default, it looks for `.js`, `.jsx`, `.ts` and `.tsx` files inside of `__tests__` folders, as well as any files with a suffix of `.test` or `.spec` (e.g. `Component.test.js` or `Component.spec.js`)
    - It will also find files called `test.js` or `spec.js`
  - Example:
    - `testRegex: ".*\.jest\.js$"`
  - Alternative: `testMatch [string | array<string>]`
    - Uses glob patterns instead of regular expressions
    - Note that one cannot specify both options

Configuring Jest – `collectCoverage`

- `collectCoverage` `[boolean]`
  - Indicates whether the coverage information should be collected while executing tests
  - Default: `false`
  - Retrofits all executed files with coverage collection statements
    - May significantly slow down tests

Configuring Jest – `coverageReporters`

- `coverageReporters [array<string|[string,options]>]`
  - A list of reporter names that Jest uses when writing coverage reports
  - Default: `["clover", "json", "lcov", "text"]`
  - Any Istanbul reporter can be used
  - Example:
    - `coverageReporters: ['html', 'lcov', ['text', { skipFull: true }]]`

Writing Tests

- `test(name, fn, timeout)` runs a test
  - `name` is the test name
  - `fn` is a function that contains the expectation to test
  - `timeout` specifies how long to wait before aborting the test
    - Optional; default: 5 seconds

```
test('my first test', () => {
  // arrange
  // act
  // assert
});
```

Grouping Tests

- `describe(name, fn)` creates a block of tests
  - Groups together several related tests
  - Can be nested

```
describe('test if', () => {
  test('true is truthy', () => {
    expect(true).toBeTruthy();
  });

  test('false is falsy', () => {
    expect(false).toBeFalsy();
  });
});
```

Parameterized Tests

- `test.each(table)(name, fn, timeout)` executes the same test with different data
  - Parameters can be positionally injected with `printf` formatting

```
test.each([
  [1, 1, 2],
  [1, 2, 3],
  [2, 1, 3],
])('%i + %i = %i', (a, b, expected) => {
  expect(a + b).toBe(expected);
});
```

```
test.each([
  { a: 1, b: 1, expected: 2 },
  { a: 1, b: 2, expected: 3 },
  { a: 2, b: 1, expected: 3 },
])('.add($a, $b)', ({ a, b, expected }) => {
  expect(a + b).toBe(expected);
});
```

```
test.each`
  a | b | expected
  ${1} | ${1} | ${2}
  ${1} | ${2} | ${3}
  ${2} | ${1} | ${3}
`('returns $expected when $a is added to $b', ({ a, b, expected }) => {
  expect(a + b).toBe(expected);
});
```

Setup and Teardown

- Jest supports setup and teardown hooks:
  - `beforeAll(fn, timeout)`
  - `beforeEach(fn, timeout)`
  - `afterEach(fn, timeout)`
  - `afterAll(fn, timeout)`

  `timeout` is optional; default: 5 seconds
- Global setup and teardown logic can be defined in Jest configuration:

`jest.config.js`:

```
globalSetup: './setup.js',
globalTeardown: './teardown.js'
```

`setup.js`:
`teardown.js`:

```
module.exports = async function (globalConfig, projectConfig) {
  // setup/teardown logic
};
```

Example: One-Time Setup

- Manage state that **can** be shared between tests:

```
beforeAll(() => {
  initializeLeberkasDatabase();
});

afterAll(() => {
  clearLeberkasDatabase();
});

test('default database has kas', () => {
  expect(get('kas')).toBeTruthy();
});

test('default database has chili-kas', () => {
  expect(get('chili-kas')).toBeTruthy();
});
```

Example: Repeating Setup

- Manage state that **cannot** be shared between tests:

```
beforeEach(() => {
  initializeLeberkasDatabase();
});

afterEach(() => {
  clearLeberkasDatabase();
});

test('default database has no kas', () => {
  expect(get('kas')).toBeFalsy();
});

test('can add kas', () => {
  expect(add('kas')).toBeTruthy();
});
```

Setup and Teardown Scopes

- Top level `before*` and `after*` hooks apply to every test in a file
- Hooks declared inside a `describe` block apply only to the tests within that `describe` block

Example: Setup and Teardown Scopes

```
beforeEach(() => {
  initializeLeberkasDatabase();
});
test('default database has kas', () => {
  expect(get('kas')).toBeTruthy();
});
test('default database has chili-kas', () => {
  expect(get('chili-kas')).toBeTruthy();
});

describe('matching leberkas to beers', () => {
  beforeEach(() => {
    initializeBeerDatabase();
  });
  test('kas + maerzen', () => {
    expect(isValidLeberkasBeerPair('kas', 'maerzen')).toBe(true);
  });
  test('chili-kas + weiss', () => {
    expect(isValidLeberkasBeerPair('chili-kas', 'weiss')).toBe(true);
  });
});
```

Applies only to tests in describe block

Example: Setup and Teardown Order

```
beforeAll(() => console.log('1 - beforeAll'));
afterAll(() => console.log('1 - afterAll'));
beforeEach(() => console.log('1 - beforeEach'));
afterEach(() => console.log('1 - afterEach'));

test('', () => console.log('1 - test'));

describe('scoped / nested block', () => {
  beforeAll(() => console.log('2 - beforeAll'));
  afterAll(() => console.log('2 - afterAll'));
  beforeEach(() => console.log('2 - beforeEach'));
  afterEach(() => console.log('2 - afterEach'));

  test('', () => console.log('2 - test'));
});
```

Output:

```
1 - beforeAll
1 - beforeEach
1 - test
1 - afterEach
2 - beforeAll
1 - beforeEach
2 - beforeEach
2 - test
2 - afterEach
1 - afterEach
2 - afterAll
1 - afterAll
```

Order of Execution

- Jest runs
  - All `describe` handlers in a test file **before** it executes any of the actual tests
    - Do setup and teardown inside `before*` and `after*` handlers rather than inside `describe` blocks
  - All tests serially in the order they were encountered in collection phase
    - Once the `describe` blocks are complete
- `before*` and `after*` hooks are called in the order of declaration
  - `after*` hooks of enclosing scope are called first

Example: Order of Execution

```
describe('describe outer', () => {
  console.log('describe outer-a');

  describe('describe inner 1', () => {
    console.log('describe inner 1');
    test('test 1', () => console.log('test 1'));
  });

  console.log('describe outer-b');
  test('test 2', () => console.log('test 2'));

  describe('describe inner 2', () => {
    console.log('describe inner 2');
    test('test 3', () => console.log('test 3'));
  });

  console.log('describe outer-c');
});
```

Output:

```
describe outer-a
describe inner 1
describe outer-b
describe inner 2
describe outer-c
test 1
test 2
test 3
```

Example: Order of Execution

```
beforeEach(() => console.log('connection setup'));
beforeEach(() => console.log('database setup'));

afterEach(() => console.log('database teardown'));
afterEach(() => console.log('connection teardown'));

test('test 1', () => console.log('test 1'));

describe('extra', () => {
  beforeEach(() => console.log('extra database setup'));
  afterEach(() => console.log('extra database teardown'));

  test('test 2', () => console.log('test 2'));
});
```

Output:

```
connection setup
database setup
test 1
database teardown
connection teardown

connection setup
database setup
extra database setup
test 2
extra database teardown
database teardown
connection teardown
```

Expectations and Matchers

- Use expecations and matchers to write assertions:

```
test('two plus two is four', () => {
  expect(2 + 2).toBe(4);
});
```

expectation matcher

- Any argument to expectation should **actual** value
- Any argument to matcher should be **expected** value
- Many matchers available to test expectation

Truethiness

- `toBeNull` matches only `null`
- `toBeUndefined` matches only `undefined`
- `toBeDefined` is the opposite of `toBeUndefined`
- `toBeTruthy` matches anything that an if statement treats as `true`
- `toBeFalsy` matches anything that an if statement treats as `false`
- Use the matcher that most precisely corresponds to what the test should check

```
test('null', () => {
  const n = null;
  expect(n).toBeNull();
  expect(n).toBeDefined();
  expect(n).not.toBeUndefined();
  expect(n).not.toBeTruthy();
  expect(n).toBeFalsy();
});

test('zero', () => {
  const z = 0;
  expect(z).not.toBeNull();
  expect(z).toBeDefined();
  expect(z).not.toBeUndefined();
  expect(z).not.toBeTruthy();
  expect(z).toBeFalsy();
});
```

Numbers

- Most ways of comparing numbers have matcher equivalents

```
test('two plus two', () => {
  const value = 2 + 2;
  expect(value).toBeGreaterThan(3);
  expect(value).toBeGreaterThanOrEqual(3.5);
  expect(value).toBeLessThan(5);
  expect(value).toBeLessThanOrEqual(4.5);

  expect(value).toBe(4);
  expect(value).toEqual(4);
});
```

- Use `toBeCloseTo` instead of `toEqual` to deal with floating point precision

```
test('adding floating point numbers', () => {
  const value = 0.1 + 0.2;
  //expect(value).toBe(0.3);
  expect(value).toBeCloseTo(0.3);
});
```

Strings

- Check strings against regular expressions with `toMatch`:

```
test('there is no I in team', () => {
  expect('team').not.toMatch(/I/);
});

test('but there is an "oliver" in brokkoliverschwendung', () => {
  expect('brokkoliverschwendung').toMatch(/oliver/);
});
```

Arrays and Iterables

- Check if array or iterable contains particular item using `toContain`:

```
const shoppingList = [
  'leberkaas',
  'beer',
  'vegetables',
  'cookies',
];

test('the shopping list has beer on it', () => {
  expect(shoppingList).toContain('beer');
  expect(new Set(shoppingList)).toContain('beer');
});
```

Errors

- Use `toThrow` to test whether a particular function throws an error:

```
function orderLeberkassemmel() {
  throw new Error('you already had one!');
}

test('ordering leberkassemmel goes as expected', () => {
  expect(() => orderLeberkassemmel()).toThrow();
  expect(() => orderLeberkassemmel()).toThrow(Error);

  expect(() => orderLeberkassemmel()).toThrow('already');
  expect(() => orderLeberkassemmel()).toThrow(/had one/);

  expect(() => orderLeberkassemmel()).toThrow(/^you already had one$/); // fails
  expect(() => orderLeberkassemmel()).toThrow(/^you already had one!$/); // passes
});
```

Matchers

- `toBe(value)`
- `toEqual(value)`
- `toBeCloseTo(number, numDigits?)`
- `toBeGreaterThan(number | bigint)`
- `toBeFalsy()`
- `toBeTruthy()`
- `toBeNull()`
- `toBeUndefined()`
- `toBeNaN()`
- `toContain(item)`
- `toBeDefined()`
- `toHaveBeenCalled()`
- `toHaveBeenCalledTimes(number)`
- `toHaveBeenCalledWith(arg1, arg2, ...)`
- `toHaveReturned()`
- `toHaveReturnedWith(value)`
- `toThrow(error?)`
- `...`

Negating Matchers

- Matchers can be negated:

```
test('two plus two is not five', () => {
  expect(2 + 2).not.toBe(5);
});
```

Marking Tests as TODO

- `test.todo(name)` marks tests that still need to be implemented:

```
test.todo('test function foobar()');
```

- Jest will highlight TODOs in test output:

```
> jest-demo@1.0.0 test
> jest --verbose

 PASS  ./example.test.js
   ✎ todo test function foobar()

 PASS  ./sum.test.js
   √ adds 1 + 2 to equal 3 (1 ms)

Test Suites: 2 passed, 2 total
Tests:       1 todo, 1 passed, 2 total
Snapshots:   0 total
Time:        0.216 s, estimated 1 s
Ran all test suites.
```

Skipping Tests

- `test.skip(name, fn)` skips a test:

```
test.skip('this test will be skipped', () => {
  throw new Error();
});
```

- Jest will reflect this in test output:

```
> jest-demo@1.0.0 test
> jest --verbose

 PASS  ./sum.test.js
  √ adds 1 + 2 to equal 3 (1 ms)

Test Suites: 1 skipped, 1 passed, 1 of 2 total
Tests:       1 skipped, 1 passed, 2 total
Snapshots:   0 total
Time:        0.347 s, estimated 1 s
Ran all test suites.
```

Testing Asynchronous Code

- Asynchronous code usually needs to be awaited, e.g., to retrieve results from asynchronous operations
- Jest needs to know when asynchronous code it should test has completed before it can move on to another test
- Jest can handle this via promises or async/await

Async Tests – Promises

- When a test returns a promise, Jest will wait for that promise to resolve
- If the promise is rejected, the test will fail

```
test('the data is peanut butter', () => {
  return fetchData().then(data => {
    expect(data).toBe('peanut butter');
  });
});
```

Async Tests – async/await

- An async test function can be used alternatively
- Within this async function, promises can be awaited

```
test('the data is peanut butter', async () => {
  const data = await fetchData();
  expect(data).toBe('peanut butter');
});
```

Always return (or await) the promise! If the `return`/`await` statement is omitted, the test will complete before the promise returned from `fetchData` resolves or rejects!

Async Tests – resolves/rejects

- When using the `.resolves` matcher, Jest will wait for that promise to resolve
- If the promise is rejected, the test will automatically fail

```
test('the data is "beer"', () => {
  return expect(fetchData()).resolves.toBe('beer');
});
```

Always return the assertion! If the `return` statement is omitted, the test will complete before the promise returned from `fetchData` resolves!

Async Tests – resolves/rejects

- When using the `.rejects` matcher, Jest will wait for that promise to be rejected
- If the promise is resolved, the test will automatically fail

```
test('the fetch fails with an error', () => {
  return expect(fetchData()).rejects.toMatch('error');
});
```

Always return the assertion! If the `return` statement is omitted, the test will complete before the promise returned from `fetchData` rejects!

Parallel Testing

- Use `test.concurrent` to run tests concurrently (within a file):
  - Considered experimental – see issues

```
test.concurrent('addition of 2 numbers', async () => {
  expect(5 + 3).toBe(8);
});

test.concurrent('subtraction 2 numbers', async () => {
  expect(5 - 3).toBe(2);
});
```

Mocking

- Mock functions allow to
  - Test the links between code by erasing the actual implementation of a function
  - Capture calls to the function (and the parameters passed in those calls)
  - Capture instances of constructor functions when instantiated with new
  - Implement test-time configuration of return values
- Two ways to mock functions:
  - Creating a mock function to use in test code
  - Writing a manual mock to override a module dependency

Mock Functions – Examples

`forEach.test.js`:

```
import { jest, test, expect } from '@jest/globals';
import { forEach } from './forEach';

const mockCallback = jest.fn(x => 42 + x);

test('forEach mock function', () => {
  forEach([0, 1], mockCallback);

  // mock function was called twice
  expect(mockCallback.mock.calls).toHaveLength(2);

  // first argument of the first call was 0
  expect(mockCallback.mock.calls[0][0]).toBe(0);

  // first argument of the second call was 1
  expect(mockCallback.mock.calls[1][0]).toBe(1);

  // return value of the first call was 42
  expect(mockCallback.mock.results[0].value).toBe(42);

  // return value of the second call was 43
  expect(mockCallback.mock.results[1].value).toBe(43);
});
```

`forEach.js`:

```
export function forEach(items, callback) {
  for (const item of items) {
    callback(item);
  }
}
```

`.mock` Property

- Mock functions have a special `.mock` property
  - Stores data about
    - How often the function was called
    - What arguments the function was called with
    - What instances of a function were created
    - What the function returned
    - What contexts a functions was called in
      - A context is the `this` value that a function receives when called

`.mock` Property – Instances and Contexts

```
const module = {
  x: 42,
  getX() { return this.x; }
};

const unboundGetX = module.getX;
console.log(unboundGetX());
// function gets invoked at the global scope
// expected output: undefined

const boundGetX = unboundGetX.bind(module);
console.log(boundGetX());
// expected output: 42
```

```
import { expect, jest, test } from '@jest/globals';

test('test instances and contexts', () => {
  const myMock1 = jest.fn();
  const a = new myMock1();
  expect(myMock1.mock.instances).toHaveLength(1);
  expect(myMock1.mock.instances[0]).toStrictEqual(a);

  const myMock2 = jest.fn();
  const b = {};
  const bound = myMock2.bind(b);
  bound();
  expect(myMock2.mock.contexts).toHaveLength(1);
  expect(myMock2.mock.contexts[0]).toStrictEqual(b);
});
```

Mocking Return Values

- Mock functions can used to inject test values into code during a test:

```
import { expect, jest, test } from '@jest/globals';

test('test return values', () => {
  const myMock = jest.fn();
  expect(myMock()).toBeUndefined();

  myMock.mockReturnValueOnce(10)
        .mockReturnValueOnce('x')
        .mockReturnValue(true);

  expect(myMock()).toStrictEqual(10);
  expect(myMock()).toStrictEqual('x');
  expect(myMock()).toStrictEqual(true);
  expect(myMock()).toStrictEqual(true);
  expect(myMock()).toStrictEqual(true);
});
```

Mock Functions

- Very effective in code that uses functional continuation-passing style:

```
import { expect, jest, test } from '@jest/globals';

test('test return values', () => {
  const filterTestFn = jest.fn();

  filterTestFn.mockReturnValueOnce(true)
              .mockReturnValueOnce(false);

  const result = [10, 20].filter(num => filterTestFn(num));
  expect(result).toStrictEqual([10]);
  expect(filterTestFn.mock.calls[0][0]).toStrictEqual(10);
  expect(filterTestFn.mock.calls[1][0]).toStrictEqual(20);
});
```

Mocking Implementations

- Specify the implementation of a mock function:

```
const myMockFn = jest.fn(cb => cb(null, true));

myMockFn((err, val) => console.log(val));
// > true
```

- Use `mockImplementationOnce` to recreate complex behavior of a mock function:

```
const myMockFn = jest
  .fn()
  .mockImplementationOnce(cb => cb(null, true))
  .mockImplementationOnce(cb => cb(null, false));

myMockFn((err, val) => console.log(val));
// > true

myMockFn((err, val) => console.log(val));
// > false
```

Mocking Implementations

- When mocked function runs out of implementations defined with `mockImplementationOnce`, it will execute the default implementation set with `jest.fn` (if it is defined):

```
const myMockFn = jest
  .fn(() => 'default')
  .mockImplementationOnce(() => 'first call')
  .mockImplementationOnce(() => 'second call');

console.log(myMockFn(), myMockFn(), myMockFn(), myMockFn());
// > 'first call', 'second call', 'default', 'default'
```

Custom Matchers

- Custom matcher functions for common forms of inspecting the `.mock` property:

```
// mock function was called at least once
expect(mockFunc).toHaveBeenCalled();

// mock function was called at least once with the specified args
expect(mockFunc).toHaveBeenCalledWith(arg1, arg2);

// last call to the mock function was called with the specified args
expect(mockFunc).toHaveBeenLastCalledWith(arg1, arg2);
```

Manual Mocks

- Manual mocks are defined by writing a module in a `__mocks__` subdirectory immediately adjacent to the module
  - To mock a module called `user` in the `models` directory, create a file called `user.js`, put it in the `models/__mocks__` directory, and **explicitly call** `jest.mock('./user')`
  - To mock a Node.js module, placed the mock in the `__mocks__` directory adjacent to `node_modules` and will be **automatically** mocked

- Examples:

```
.
├── config
├── __mocks__
│   └── fs.js
├── models
│   ├── __mocks__
│   │   └── user.js
│   └── user.js
├── node_modules
└── views
```

Manual Mocks – Example

`FileSummarizer.js`:

```
const fs = require('fs');

function summarizeFilesInDirectorySync(directory) {
  return fs.readdirSync(directory).map(fileName => ({
    directory,
    fileName,
  }));
}

exports.summarizeFilesInDirectorySync = summarizeFilesInDirectorySync;
```

Manual Mocks – Example

`__mocks__/fs.js`:

```
const path = require('path');
const fs = jest.createMockFromModule('fs');

let mockFiles = Object.create(null);
function __setMockFiles(newMockFiles) {
  mockFiles = Object.create(null);
  for (const file in newMockFiles) {
    const dir = path.dirname(file);
    if (!mockFiles[dir]) {
      mockFiles[dir] = [];
    }
    mockFiles[dir].push(path.basename(file));
  }
}

function readdirSync(directoryPath) {
  return mockFiles[directoryPath] || [];
}

fs.__setMockFiles = __setMockFiles;
fs.readdirSync = readdirSync;

module.exports = fs;
```

`__tests__/FileSummarizer-test.js`:

```
jest.mock('fs');

describe('listFilesInDirectorySync', () => {
  const MOCK_FILE_INFO = {
    '/path/to/file1.js': 'console.log("file1 contents");',
    '/path/to/file2.txt': 'file2 contents',
  };

  beforeEach(() => {
    require('fs').__setMockFiles(MOCK_FILE_INFO);
  });

  test('includes all files in the directory in the summary', () => {
    const FileSummarizer = require('../FileSummarizer');
    const fileSummary =
      FileSummarizer.summarizeFilesInDirectorySync('/path/to');

    expect(fileSummary.length).toBe(2);
  });
});
```

Spying on Constructors

- Test whether class constructor is called with the correct parameters
  - Replace the function returned by the higher-order function (HOF) with a Jest mock function:

```
import SoundPlayer from './sound-player';

jest.mock('./sound-player', () => {
  return jest.fn().mockImplementation(() => {
    return { playSoundFile: () => { } };
  });
});
```

Mocking Non-Default Class Exports

- If class to be mocked is not the default export from the module
  - Return an object with the key that is the same as the class export name

```
import { SoundPlayer } from './sound-player';

jest.mock('./sound-player', () => {
  return {
    SoundPlayer: jest.fn().mockImplementation(() => {
      return { playSoundFile: () => { } };
    }),
  };
});
```

Support for ECMAScript Modules

- Jest ships with experimental support for ECMAScript Modules (ESM)
  - The implementation may have bugs and lack features
  - Node.js APIs used to implement ESM support are still considered experimental
    - As of Node.js 18.8.0

- How to activate ESM support in tests
  - Disable code transforms or otherwise configure the transformer to emit ESM rather than the default CommonJS
  - Execute node with `--experimental-vm-modules`

Activate ESM Support (1)

- Disable code transforms:

`jest.config.js`:

```
/** @type {import('jest').Config} */
const config = {
  // ...
  transform: {},
  // ...
};

export default config;
```

Activate ESM Support (2)

- Execute node with `--experimental-vm-modules`:

`package.json`:

```
{
  // ...
  "scripts": {
    "test": "node --experimental-vm-modules↩
               --disable-warning=ExperimentalWarning node_modules/jest/bin/jest.js"
  }
}
// or
{
  // ...
  "scripts": {
    "test": "NODE_OPTIONS=\"$NODE_OPTIONS --experimental-vm-modules↩
               --disable-warning=ExperimentalWarning\" npx jest"
  }
}
```

Module Mocking in ESM

- ESM evaluates static import statements before looking at the code
  - Hoisting of `jest.mock` calls that happens in CJS won't work for ESM
  - Calls to `require()` or `import()` must be made after `jest.mock` calls to load mocked modules
  - Same applies to modules which load the mocked modules
- ESM (un-)mocking is supported via two functions:
  - `jest.unstable_mockModule()`
    - Essentially the same as `jest.mock` with two differences
      - Factory function is required and it can be sync or async
  - `jest.unstable_unmockModule()`
  - API is still work in progress

Module (Un-)mocking in ESM – Example

`esm-module.test.js`:

```
import { jest, test, expect } from '@jest/globals';

test('test esm-module', async () => {
  jest.unstable_mockModule('./esm-module.js', () => ({
    default: () => 'default mocked 1st time',
    namedFn: () => 'namedFn mocked 1st time',
  }));

  const firstMockModule = await import('./esm-module.js');
  expect(firstMockModule.default()).toBe('default mocked 1st time');
  expect(firstMockModule.namedFn()).toBe('namedFn mocked 1st time');

  jest.unstable_unmockModule('./esm-module.js');

  const originalModule = await import('./esm-module.js');
  expect(originalModule.default()).toBe('default');
  expect(originalModule.namedFn()).toBe('namedFn');

  /* mocked module cached, won't override */
  jest.unstable_mockModule('./esm-module.js', () => ({
    default: () => 'default mocked 2nd time',
    namedFn: () => 'namedFn mocked 2nd time',
  }));

  const secondMockModule = await import('./esm-module.js');
  expect(secondMockModule.default()).not.toBe('default mocked 2nd time');
  expect(secondMockModule.default()).toBe('default mocked 1st time');
  expect(secondMockModule.namedFn()).not.toBe('namedFn mocked 2nd time');
  expect(secondMockModule.namedFn()).toBe('namedFn mocked 1st time');
});
```

`esm-module.js`:

```
export default () => {
  return 'default';
};

export const namedFn = () => {
  return 'namedFn';
};
```

Module (Un-)mocking in ESM – Example

`esm-module.test.js`:

```
import { jest, test, expect } from '@jest/globals';

test('test esm-module', async () => {
  jest.unstable_mockModule('./esm-module.js', () => ({
    default: () => 'default mocked 1st time',
    namedFn: () => 'namedFn mocked 1st time',
  }));

  const firstMockModule = await import('./esm-module.js');
  expect(firstMockModule.default()).toBe('default mocked 1st time');
  expect(firstMockModule.namedFn()).toBe('namedFn mocked 1st time');

  jest.unstable_unmockModule('./esm-module.js');

  const originalModule = await import('./esm-module.js');
  expect(originalModule.default()).toBe('default');
  expect(originalModule.namedFn()).toBe('namedFn');

  jest.resetModules();
  jest.unstable_mockModule('./esm-module.js', () => ({
    default: () => 'default mocked 2nd time',
    namedFn: () => 'namedFn mocked 2nd time',
  }));

  const secondMockModule = await import('./esm-module.js');
  expect(secondMockModule.default()).not.toBe('default mocked 1st time');
  expect(secondMockModule.default()).toBe('default mocked 2nd time');
  expect(secondMockModule.namedFn()).not.toBe('namedFn mocked 1st time');
  expect(secondMockModule.namedFn()).toBe('namedFn mocked 2nd time');
});
```

`esm-module.js`:

```
export default () => {
  return 'default';
};

export const namedFn = () => {
  return 'namedFn';
};
```

Snapshot Testing

- Very useful to make sure UI does not change unexpectedly
  - Test renders UI component and takes a snapshot
  - On first run, reference snapshot file is created and stored alongside the test
  - On subsequent runs, compares new snapshot to reference snapshot
  - Test will fail if the two snapshots do not match
    - Either the change is unexpected
    - Or reference snapshot needs to be updated to the new version of the UI component

Snapshot Testing – First Execution

`tests/snapshot.test.js`:

```
/**
 * @jest-environment jsdom
 */

test('snapshot testing example', async () => {
  const div = document.createElement('span');
  div.innerHTML = new Date().toISOString();
  expect(div).toMatchSnapshot();
});
```

first execution creates snapshot file

`tests/__snapshots__/snapshot.test.js.snap`:

```
// Jest Snapshot v1, https://jestjs.io/docs/snapshot-testing

exports[`snapshot testing example 1`] = `
<span>
  2026-03-15T09:06:56.738Z
</span>
`;
```

```
❯ npm test

> frontend@1.0.0 test
> node --experimental-vm-modules↩
    --disable-warning=ExperimentalWarning↩
    node_modules/jest/bin/jest.js

 PASS  tests/CounterComponent.shadow-dom.test.js
 PASS  tests/testing-library.test.js
 PASS  tests/snapshot.test.js
  › 1 snapshot written.
 PASS  tests/Math.test.js
 PASS  tests/CounterComponent.test.js
 PASS  tests/CounterComponent.alt.test.js

Snapshot Summary
  › 1 snapshot written from 1 test suite.

Test Suites: 6 passed, 6 total
Tests:       20 passed, 20 total
Snapshots:   1 written, 5 passed, 6 total
Time:        1.031 s
Ran all test suites.
```

Snapshot Testing – Second Execution

```
FAIL tests/snapshot.test.js
  ● snapshot testing example

    expect(received).toMatchSnapshot()

    Snapshot name: `snapshot testing example 1`

    - Snapshot  - 1
    + Received  + 1

      <span>
    -   2026-03-15T09:06:56.738Z
    +   2026-03-15T09:20:02.937Z
      </span>

       6 |   const div = document.createElement('span');
       7 |   div.innerHTML = new Date().toISOString();
      > 8 |   expect(div).toMatchSnapshot();
         |               ^
       9 | });

      at Object.<anonymous> (tests/snapshot.test.js:8:15)

 › 1 snapshot failed.
Snapshot Summary
 › 1 snapshot failed from 1 test suite. Inspect your↩
   code changes or run `npm test -- -u` to update them.

Test Suites: 1 failed, 5 passed, 6 total
Tests:       1 failed, 19 passed, 20 total
Snapshots:   1 failed, 5 passed, 6 total
Time:        1.202 s
Ran all test suites.
```

second execution fails because snapshot taken during this test execution do not match previously stored snapshot; if changes are expected, update snapshot via `npm test -- -u`; snapshots can also be interactively updated in watch mode via `npm test -- --watch`

Inline Snapshots

- Writes snapshot values back into test code:

```
/**
 * @jest-environment jsdom
 */

test('snapshot testing example', async () => {
  const div = document.createElement('span');
  div.innerHTML = new Date().toISOString();
  expect(div).toMatchInlineSnapshot();
});
```

`npm test`

```
/**
 * @jest-environment jsdom
 */

test('snapshot testing example', async () => {
  const div = document.createElement('span');
  div.innerHTML = new Date().toISOString();
  expect(div).toMatchInlineSnapshot(`
<span>
2026-03-15T09:52:02.657Z
</span>
`);
});
```